Day 1 — Senior Track

Python you *think* you know

For engineers who write Python daily.
Pull these in when the room is ahead — interleave with the base deck per module, or run as a block before the labs.

🎓 SENIOR TRACK

How to use this deck

- These slides **supplement** the base Day-1 deck — they don't replace it.
- Map: **M1 → after base Module 1**, **M2 → after base Module 2**, **M3 → after base Module 3**.
- Each module ends with a **lab extension** (`README-senior.md` inside each lab folder) that fast finishers do *instead of* the base stretch goals.
- Everything still lands on `checkpoints/day-2-start/` — the senior path is a *deeper* route to the same Day-2 baseline, not a fork.

🎓 SENIOR TRACK

Module 1 — Core, the parts that bite

The base module covers shapes. Here's what separates code that *works* from code that *survives*:

- **EAFP > LBYL** — Python idiom is `try/except`, not `if check: ...`
- **Exceptions chain** — `raise X from Y` keeps the cause
- **`else` / `finally` on `try`** have precise, often-misused semantics
- **Mutable defaults** are evaluated once, at def-time
- **Closures** capture *variables*, not values — the classic loop bug

🎓 SENIOR TRACK

EAFP vs LBYL

```python
# LBYL — racy, verbose, double lookup
if product_id in catalog._items:
    return catalog._items[product_id]
raise CatalogError( ... )

# EAFP — one lookup, idiomatic, no TOCTOU gap
try:
    return catalog._items[product_id]
except KeyError:
    raise CatalogError(f"id {product_id} not found") from None
```

`from None` suppresses the `KeyError` context when it's noise.

`from exc` *keeps* it when the cause matters. Choose deliberately.

🎓 SENIOR TRACK

try / else / finally — the exact rules

```
try:
    product = catalog.get(pid)
except CatalogError:
    log.warning("miss")          # runs only on exception
else:
    audit(product)               # runs only if NO exception
finally:
    release_lock()               # always runs, even on return/raise
```

- `else` exists so the "success-only" code isn't wrapped in the `try` (and can't accidentally catch *its* exceptions).
- `finally` runs even if `try` contains `return` — a `return` in `finally` will *swallow* a propagating exception. Don't.

🎓 SENIOR TRACK

The two traps that survive code review

```
# 1. Mutable default — shared across ALL calls
def add(self, product, tags=[]):   # ⚠️ one list, forever
    tags.append("new"); ...        # leaks between calls

# 2. Late-binding closure
makers = [lambda: i for i in range(3)]
[m() for m in makers]              # [2, 2, 2], not [0, 1, 2]
```

Fixes: `tags=None` then `tags = tags or []`; and `lambda i=i: i` to bind now.

These are *language semantics*, not bugs — which is why they pass tests until they don't.

🎓 SENIOR TRACK

When a generator beats a list

```
# materializes everything — fine for 5 products, fatal for 5M rows
rows = [parse(line) for line in open("huge.csv")]

# lazy — constant memory, starts work immediately
rows = (parse(line) for line in open("huge.csv"))
total = sum(r.price for r in rows)   # one pass, nothing held
```

Rule of thumb: if you only iterate **once** and don't need `len()` or indexing, use a generator. Day 1 catalog is tiny — but the bulk-import on Day 2 is where this pays off.

→ **Lab 1+** (`lab-01-product-foundation/README-senior.md`): frozen/hashable `Product`, `__lt__` sorting, `Counter`-based `summary()`.

🎓 SENIOR TRACK

Module 2 — `collections` & the stdlib you skip

```python
from collections import defaultdict, Counter, namedtuple

# group_by in one line, no KeyError
groups = defaultdict(list)
for p in products: groups[p.category].append(p)

# count_by_category in one line
counts = Counter(p.category for p in products)
# Counter({'Electronics': 3, 'Fitness': 1})
```

`Counter` *is* the `group_by_category` → count pipeline. Day 4's agent exposes exactly this as a tool. Reach for stdlib before writing loops.

🎓 SENIOR TRACK

Context managers — write one

```python
from contextlib import contextmanager
import os, tempfile, pathlib

@contextmanager
def atomic_write(path: pathlib.Path):
    tmp = path.with_suffix(path.suffix + ".tmp")
    fh = tmp.open("w")
    try:
        yield fh
        fh.close()
        os.replace(tmp, path)     # atomic on POSIX + Windows
    except BaseException:
        fh.close(); tmp.unlink(missing_ok=True)
        raise
```

A half-written `catalog.json` after a crash is data loss. `os.replace` makes the save all-or-

🎓 SENIOR TRACK

`pathlib` over `os.path`, dict ordering

```
from pathlib import Path
data = Path("catalog.json").read_text()        # no open/close
for f in Path("data").glob("*.csv"): ...        # no os.listdir + join

# dicts preserve insertion order (guaranteed 3.7+)
# — so JSON round-trips stable, and dict(Counter.most_common()) is sorted
```

- `Path` objects compose with `/`: `Path("data") / "products.csv"`.
- Insertion-order is a *language guarantee* now — rely on it for stable output, but never for "sorted" (it isn't).

→ **Lab 2+** (`lab-02-persistent-catalog/README-senior.md`): `atomic_write` save, generator-streamed CSV load, `defaultdict` / `Counter` queries, full `pathlib`.

🎓 SENIOR TRACK

Module 3 — OOP beyond `__init__`

```python
@dataclass(frozen=True, slots=True)
class Product:
    id: int
    name: str
    price: float
```

- `frozen=True` → immutable + auto `__hash__` → usable in `set` / `dict` keys
- `slots=True` → no per-instance `__dict__`: less memory, faster attr access, typo-on-assign raises
- `__post_init__` for cross-field validation (the dataclass equivalent of Pydantic's validators — the Day-2 bridge)

🎓 SENIOR TRACK

Dunders that earn their keep

```python
@dataclass(frozen=True, order=True)   # order= gives __lt__ etc.
class Product:
    id: int; name: str; price: float

sorted(catalog.list_all())            # works — sorts by field order
heapq.nlargest(3, products, key=lambda p: p.price)
```

- `order=True` generates `__lt__/__le__/__gt__/__ge__` from field order.
- `frozen=True` generates `__hash__` + `__eq__`.
- Define `__hash__` / `__eq__` *consistently* — two equal objects MUST hash equal, or sets break silently.

🎓 SENIOR TRACK

`functools` — the decorator toolkit

```python
from functools import lru_cache, partial, singledispatch, wraps

@lru_cache(maxsize=256)
def categorize(price: float) → str: ...    # memoized, thread-safe

to_json = partial(json.dumps, indent=2, sort_keys=True)

@singledispatch
def render(x): ...                          # dispatch on arg type
@render.register
def _(x: Product): return x.name
```

`@wraps` (which you already use in `@retry`) is *why* `functools.wraps` matters — without it, pytest can't introspect the wrapped function on Day 3.

🎓 SENIOR TRACK

Stateful & parametrized decorators

```
def retry(times=3, delay=0.1, backoff=2.0):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            wait = delay
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except Exception:
                    if attempt == times: raise
                    time.sleep(wait); wait *= backoff   # exponential
        return wrapper
    return decorator
```

The base `@retry` uses a fixed delay. **Exponential backoff** is what you actually want against a flaky API (Day 2's `APIClient`). Three nested functions — the shape never changes.

🎓 SENIOR TRACK

`Protocol` — typing without inheritance

```python
from typing import Protocol

class SupportsToDict(Protocol):
    def to_dict(self) -> dict: ...

def save_json(items: list[SupportsToDict], path): ...
```

- Structural typing: *anything* with `to_dict()` satisfies it — no base class, no import coupling.
- This is how you type "duck-typed" code honestly. Day 4's tool registry uses the same idea: a tool is *anything callable matching a signature*, not a subclass.

🎓 SENIOR TRACK

FastAPI dependency injection

```python
from fastapi import Depends

def get_catalog() -> ProductCatalog:
    return app.state.catalog            # one place to swap impl

@app.get("/products")
def list_products(cat: ProductCatalog = Depends(get_catalog)):
    return [p.to_dict() for p in cat.list_all()]
```

The base server uses a module-global `catalog`. `Depends` makes it **injectable** — which is exactly what lets Day 3 swap a fresh catalog per test without monkeypatching globals. DI on Day 1 = clean fixtures on Day 3.

→ **Lab 3+** (`lab-03-local-api-server/README-senior.md`): `Depends`-injected catalog, `@property` computed fields, exponential-backoff `@retry`, typed responses.

🎓 SENIOR TRACK

End of Day 1 — Senior Track ✅

You took the deeper route to the **same** `day-2-start` baseline:

- Immutable, hashable, orderable `Product`
- Atomic persistence + generator-streamed I/O
- `functools` + stateful decorators + `Protocol` typing
- A DI-clean FastAPI server ready for painless Day-3 tests

Tomorrow's Pydantic + `APIClient` will feel like a small step, not a leap.